

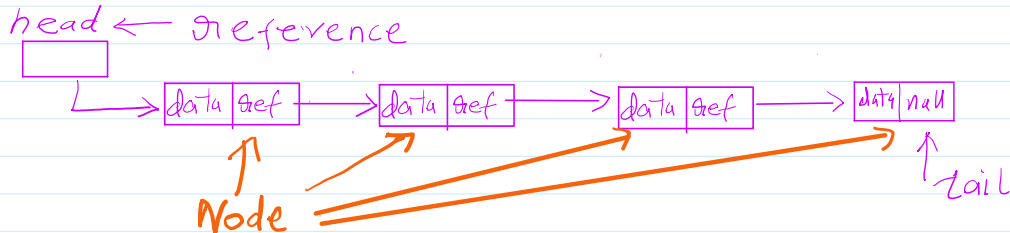
Linked List Data Structure

10 August 2026 19:52

A linked list is a linear data structure that includes a series of connected nodes. A linked list can be defined as a set of nodes that are randomly stored in memory. A node in the linked list contains 2 parts, that is the first is the data part and 2nd is the reference part. The last node of the list contains a pointer to NULL. After array linked list is the second most used data structure. In a linked list, every link contains a connection to another link.

Representation of a linked list

A linked list can be represented as a connection of nodes in which each node points to the next node of the list.

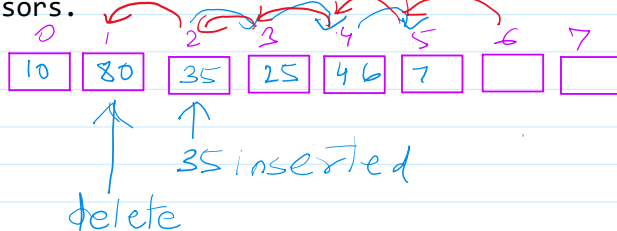


Till now we have been using an array data structure to organize a group of elements that are to be stored individually in the memory. However, array has several advantages and disadvantages that must be known to decide the data structure that will be used throughout the program.

Why use a linked list over an Array?

Structure that overcomes the limitations of arrays. Let's first see some of the limitations of arrays:

- The size of the array must be known in advance before using it in the program.
- Increasing the size of array is a time consuming process. It is almost impossible to expand the size of the array at runtime.
- All the elements in the array need to be contiguously stored in the memory. Inserting or deleting an element in the array meets shifting of all its predecessors.



The linked list is useful because:

- It allocates the memory dynamically, all the nodes of the linked list are non-contiguously stored in the memory and linked together with the help of reference variables.
- In a linked list, size is no longer a problem since we do not need to define its size at the time of declaration. The list grows as per the programs demands, and it reduces its size as per requirement.

Structure of a Node

```
package DataStructures.LinkedListsInJava;
```

```

/**
 * Write a description of class Node here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Node
{
    private int data;
    private Node next;

    public Node(int data){
        this.data = data;
        next = null;
    }

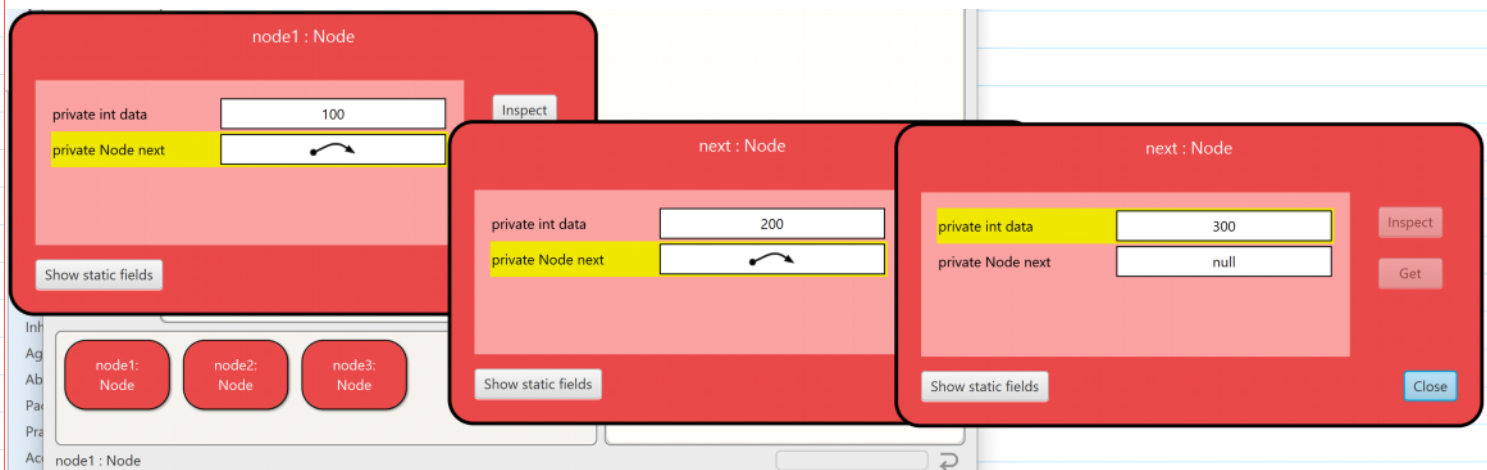
    public int getData(){
        return data;
    }

    public void setData(int data){
        this.data = data;
    }

    public void setNext(Node n){
        this.next = n;
    }
    /**
     * @return Returns the current node reference
     */
    public Node getNode(){
        return next;
    }

    public String toString(){
        return "==>"+data+" ";
    }
}

```



```

Node head = node1;
while(head != null){
    System.out.print(head);
    head = head.getNode();
}

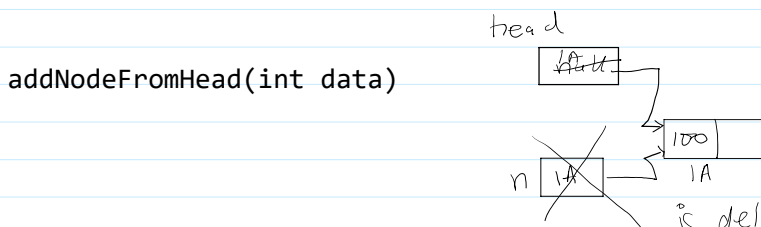
```

Output:
 ==>100 ==>200 ==>300

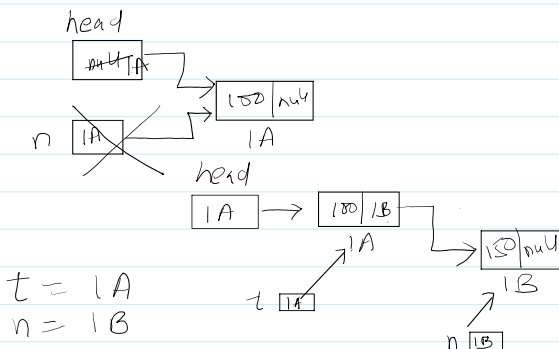
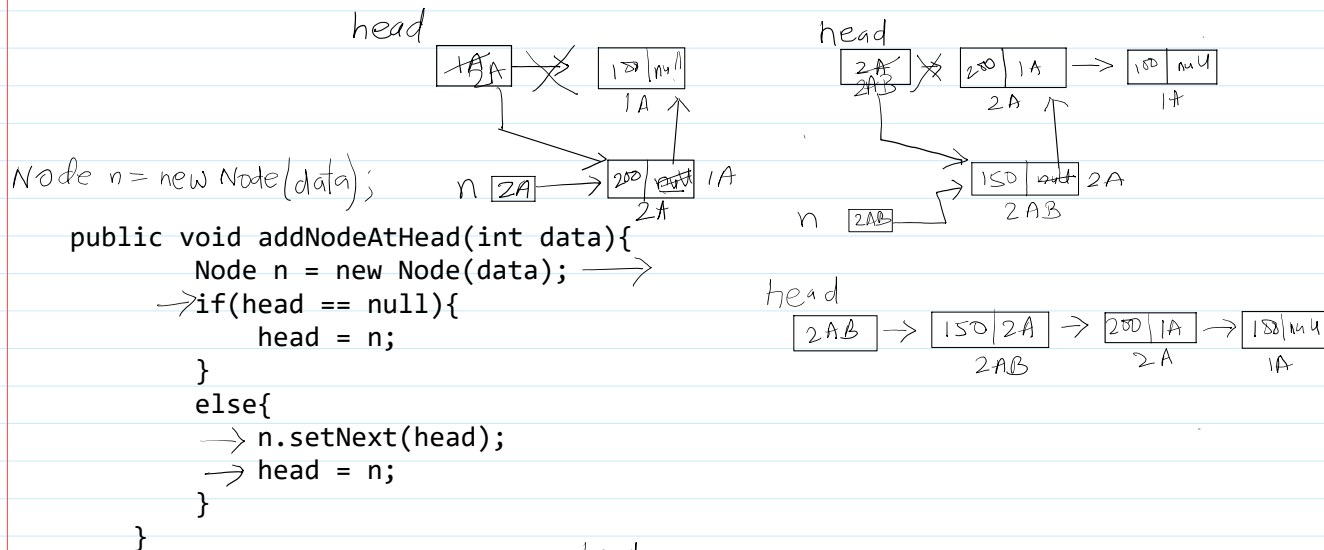
What is head?

Instead of keeping separate variables such as node1, node2 and node3. We generally maintain one reference variable called **head**.
head point to the first node.

Initially, head = null; which means the linked list is empty.



is deleted because local variables or objects are collected by garbage collector, when program control goes back to the caller of the method.



```

public void addNodeAtEnd(int data){
    head
    1A -> 100 1B -> 150 1C

```

```
public void addNodeAtEnd(int data){
```

```
    Node n = new Node(data);
```

```
    if(head == null){
```

```
        head = n;
```

```
    }
```

```
    else{
```

```
        Node t = head;
```

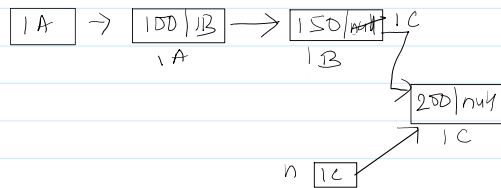
```
        //loop used to point the last node of the linked list
```

```
        for(;t.getNext() != null; t.setNext(t.getNext()));
```

```
    } t.setNext(n); //attaching the node at end;
```

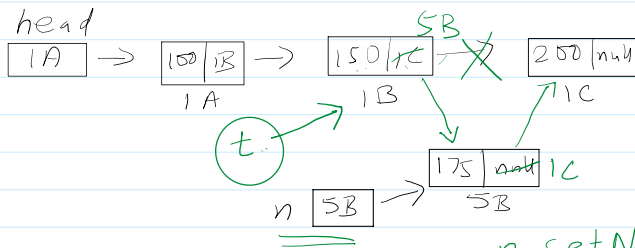
```
}
```

head



t = 1A 1B

after = 150



n.setNext(t.getNext());
t.setNext(n);

```
public void addNodeBeforeANode(int data, int before){
```

```
    if(head == null){
```

```
        System.out.print("\nList is empty");
```

```
        return;
```

```
    }
```

```
    for(Node p = null, t = head; t != null; t = t.getNext()){
```

```
        if(t.getData() == before){
```

```
            Node n = new Node(data);
```

```
            n.setNext(t);
```

```
            if(p == null){ head = n; return; }
```

```
            p.setNext(n);
```

```
        } return;
```

```
    }
```

```
    p = t;
```

```
    }
```

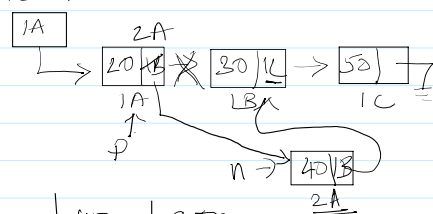
```
    System.out.print("\n" + before + " is not present in the list.");
```

addNodeBeforeANode(100, 20);

addNodeBeforeANode(200, 50);

addNodeBeforeANode(500, 21);

head



data: 40

before: 30

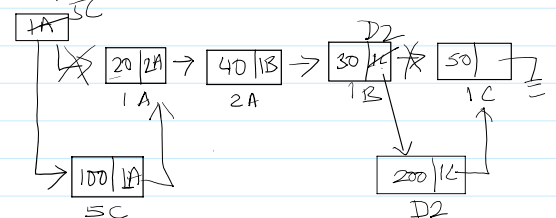
t = 1A 1B

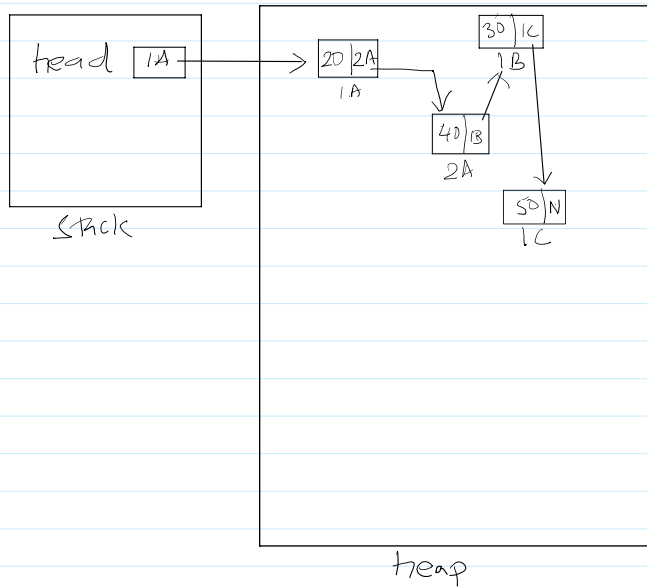
n = 2A

p = 1A

100	200	500
20	50	21
1A	5C 1A 2A 1B 1C	5C 1A 2A 1B 2C
5C	D2	
null	null 5C 1A 2A 1B	null 5C 1A 2A 1B D2 1C

head





```

public void addNodeAfterANode(int data, int after)2000 {1000
    → if(head == null){
        System.out.print("\nList is empty");
        return; //exit from the method
    }
    for(Node t = head; t != null; t = t.getNext())1A {→ 1B 1C 1D 1E 1F 2A
        if(t.getData() == after)210 {↗
            → Node n = new Node(data);
            n.setNext(t.getNext());
            → t.setNext(n);
            → return ; //exit from the method
        }
    }
    System.out.print("\n" + after + " is not present in the list.");
}

```

Linked list: ==>^{head 1A}100 ==>10 ==>200 ==>30 ==>50 ==>500 ==>1000 ==>null

1A 1B 1C 1D 1E 1F 2A

2B